On the container level (C4 model), solutions are not much different. With stream processing, we use stream ports from source data products, and our Fraud Prediction container is doing predictions using a stream-processing mechanism. With batch processing, we are using files and a batch-processing mechanism.

We will not perform a tradeoff analysis for these two options, but your main driver would be response time, and the measure of success for this driver would guide the decision. If a nearer real-time response time is needed than stream processing, or if a longer cadence is possible, think about using the batch-processing mechanism.

But what is more interesting for us in terms of a data mesh is to find answers to the following questions:

1. How do we expose streams as a data product?
2. How should data products take advantage of a data lake solution?

Let's start with event streams.

EVENT STREAM AS A DATA PRODUCT PORT

In the first design, we need to feed a stream-processing engine with three streams of events from three different systems. We will describe two ways to expose these events:

- Extract events from files by using an engine like Airflow.
- Use the outbox pattern.

The outbox pattern

The *outbox pattern* (http://mng.bz/06DN), also known as the transactional outbox pattern, is used whenever we want to change the state of the database and publish events or messages consistently. This pattern takes advantage of a native database mechanism: the transaction.